인공지능과 기계학습

2주차

2026.03.11

장우영

목 차

1. 사이킷런을 타고 떠나는 머신 러닝 분류 모델 투어

3. 사이킷런을 타고 떠나는 머신 러닝 분류 모델 투어

3.1 분류 알고리즘 선택
3.2 사이킷런 첫걸음: 퍼셉트론 훈련
3.3 로지스틱 회귀를 사용한 클래스 확률 모델링
3.4 서포트 벡터 머신을 사용한 최대 마진 분류

학습 목표

- 강력하고 인기 있는 분류 알고리즘 소개
  - 로지스틱 회귀
  - 서포트 벡터 머신
  - 결정 트리
  - K 최근접 이웃
- 사이킷런 머신러닝 라이브러리 사용하기
- 분류 알고리즘의 강점과 약점 설명

3.1 분류 알고리즘 선택

- 특정 문제에 알맞은 분류 알고리즘을 선택하려면 연습과 경험이 필요
  - 실제로 최소한 몇 개의 학습 알고리즘 성능을 비교하고, 해당 문제에 최선인 모델을 선택하는 것이 권장됨
- 알고리즘을 훈련하기 위한 다섯 가지 주요 단계
  1. 특성을 선택하고 훈련 샘플 수집
  2. 성능 지표를 선택
  3. 학습 알고리즘 선택, 모델 훈련
  4. 모델 성능 평가
  5. 알고리즘 설정 변경 및 모델 튜닝

3.2 사이킷런 첫걸음: 퍼셉트론 훈련 1/2

- 데이터셋
  - 150개의 꽃 샘플
  - 꽃잎 길이와 꽃잎 너비를 특성 행렬 X에 할당
  - 꽃 품종에 해당하는 클래스 레이블을 벡터 y에 할당
- 훈련 데이터셋, 테스트 데이터셋
  - test_size = 0.3 # 테스트 비율
  - Random-state = 1 # 고정된 랜덤 시드
  - Stratify y # 계층화 기능
- 전처리
  - StandardScaler

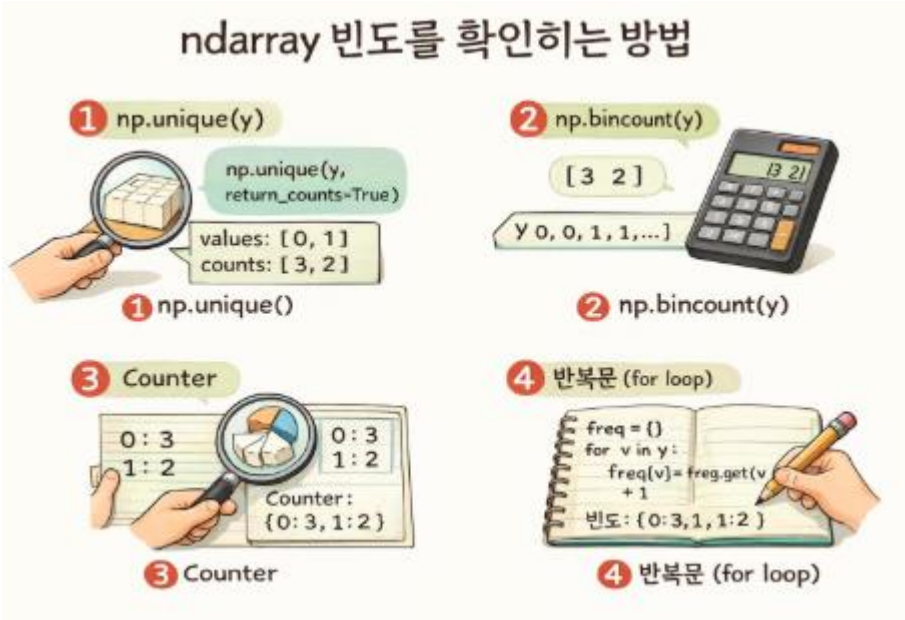
3.2 사이킷런 첫걸음: 퍼셉트론 훈련 2/2

- 퍼셉트론 모델 훈련
  - Eta: 0.1
- 테스트
  - 성능 평가

자료 구조 확인 type

- type(10) # int
- type(3.14) # float
- type("hello") # str
- type([1,2,3]) # list
- type((1,2,3)) # tuple
- type({1,2,3}) # set
- type({"a":1}) # dict
- numpy # ndarry
- Pandas # DataFrame
- pytorch # Tensor

두부를 자르는 도구 / ndarray 빈도 확인 방법



3.2 사이킷런 첫걸음: 퍼셉트론 훈련

```
# 데이터셋
from sklearn import datasets

iris = datasets.load_iris()
X = iris.data[:, [2, 3]]
y = iris.target
```

3.2 사이킷런 첫걸음: 퍼셉트론 훈련

```
# 훈련  데이터셋, 테스트 데이터셋
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
     y,
    test_size=0.3,
    random_state=1,
    stratify=y
)
```

3.2 사이킷런 첫걸음: 퍼셉트론 훈련

```
# 전처리
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(X_train)
X_train_std = sc.transform(X_train)
X_test_std = sc.transform(X_test)
```

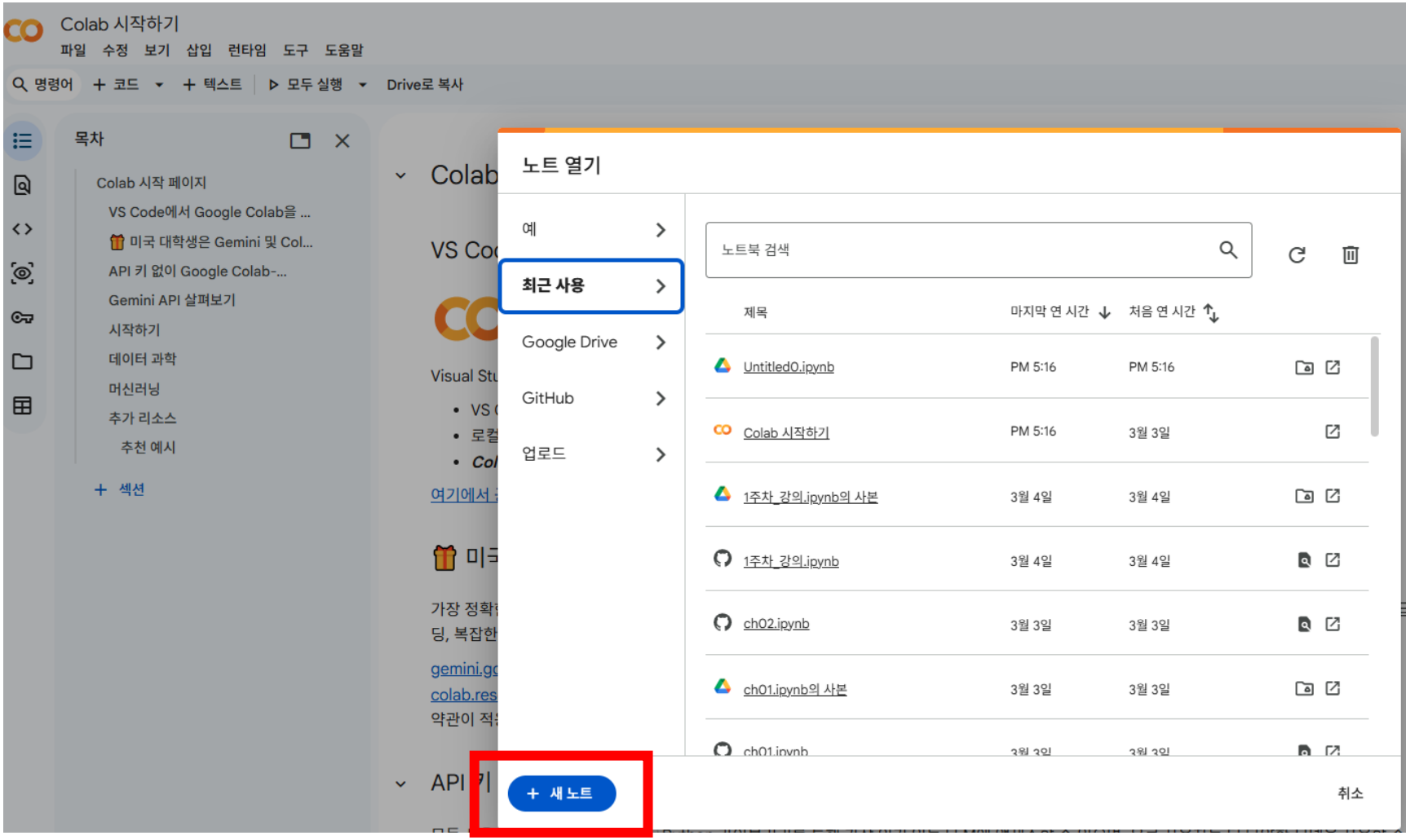
3.2 사이킷런 첫걸음: 퍼셉트론 훈련

```
# 퍼셉트론 훈련
from sklearn.linear_model import Perceptron

ppn = Perceptron(
  eta0=0.1,
  random_state=1
)
ppn.fit(X_train_std, y_train)
```

3.2 사이킷런 첫걸음: 퍼셉트론 훈련

```
# 테스트
y_pred = ppn.predict(X_test_std)
print("테스트 샘플:",len(y_pred))
print("오분류 샘플: %d" % (y_test != y_pred).sum())

# 성능평가
from sklearn.metrics import accuracy_score
print('정확도: %.2f' % accuracy_score(y_test, y_pred))

print('정확도: %.2f' % ppn.score(X_test_std, y_test))
```

3.2 사이킷런 첫걸음: 퍼셉트론 훈련 (실습)



3.3 로지스틱 회귀를 사용한 클래스 확률 모델링

- 퍼셉트론은 머신 러닝 분류 알고리즘을 배우기 좋은 모델
  - 선형적으로 구분되지 않을 때 수렴하지 않음
  - 에포크 마다 적어도 하나의 샘플이 잘못 분류되면,
    가중치 업데이트가 끝도 없이 계속

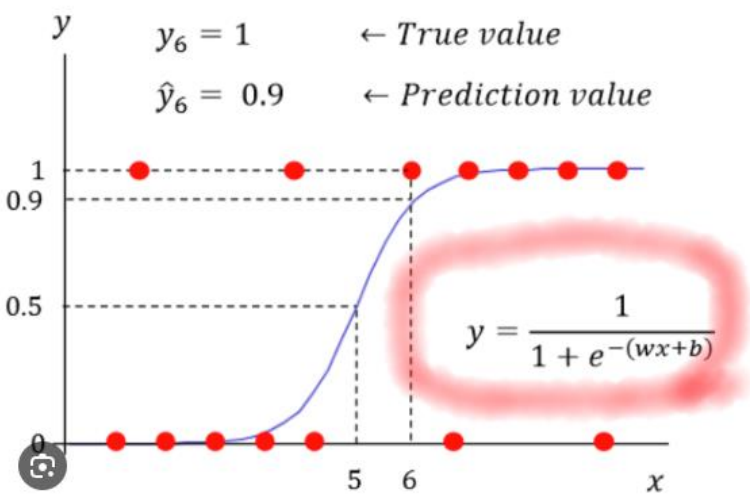
- 선형 이진 분류 문제에 다른 알고리즘 “로지스틱 회귀”
  - 분류 모델

3.3.1 로지스틱 회귀의 이해와 조건부 확률

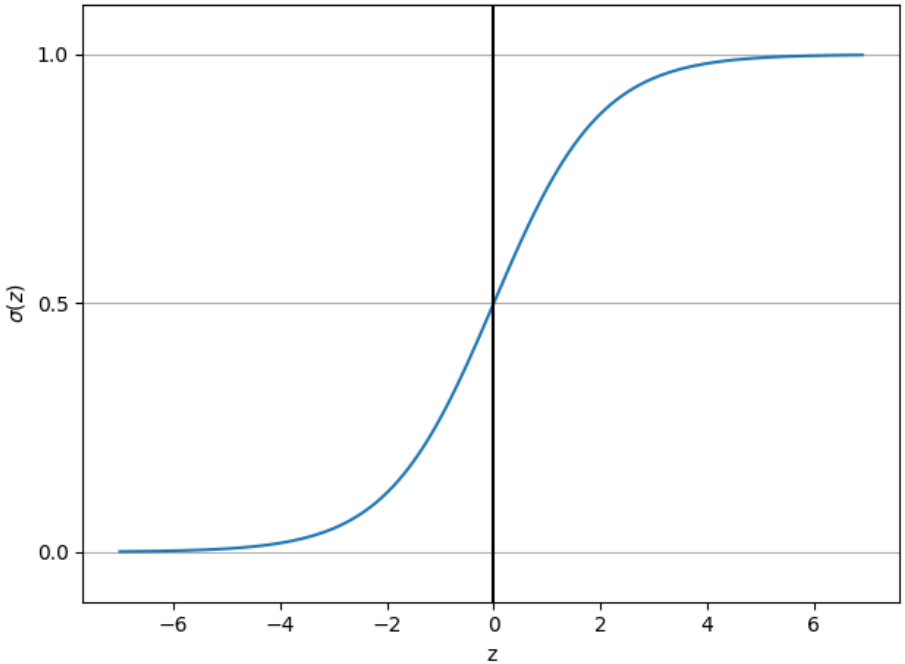
- 이진 분류를 위한 선형 모델
  - 로지스틱 회귀 모델 이면에 있는 아이디어
  - 오즈비, 자연 log ,0~1 사이의 값을 갖는 logit 함수
  - 로지스틱 시그모이드 함수: 함수모양이 S자 형태를 띠기 때문
- 다중 클래스를 위한 로지스틱 회귀

  (다항 로지스틱 회귀, 소프트맥스 회귀)

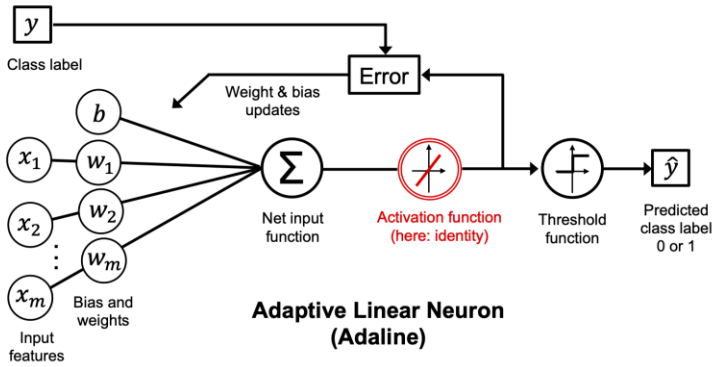
3.3.1 로지스틱 회귀의 이해와 조건부 확률

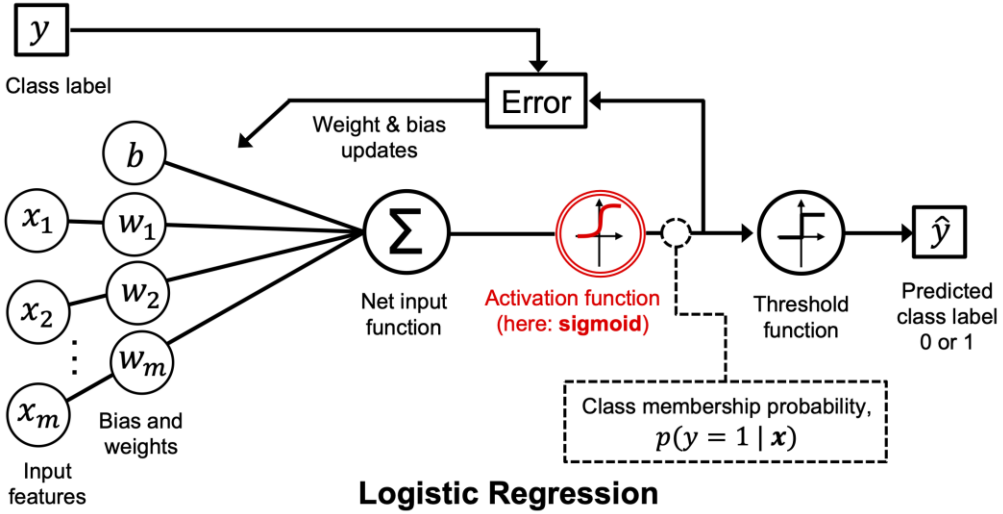


3.3.1 로지스틱 회귀의 이해와 조건부 확률



실수 입력 값을 [0,1] 사이의 값으로 변환
중간은 0.5





3.3.2 로지스틱 손실 함수의 가중치 학습

- 정답에 가까운 확률을 주면 벌점이 작고 틀린 확률을 주면 벌점이 크게 증가하는 함수

구분	퍼셉트론 (Perceptron)	아달린 (Adaline)	로지스틱 회귀 (Logistic Regression)
모델 형태	선형 모델	선형 모델	선형 모델
출력값	0 또는 1	연속값 (실수)	확률 (0~1)
활성화 함수	Step 함수	없음 (선형 출력)	Sigmoid 함수
결정 방식	임계값 기반 분류	선형값 계산 후 분류	확률 기반 분류
학습 기준	분류 오류	평균제곱오차(MSE)	Log Loss
학습 방식	틀린 샘플만 수정	전체 데이터 오차 최소화	확률 오차 최소화
미분 가능 여부	Step 함수 → 미분 불가	가능	가능
최적화 방법	규칙 기반 업데이트	Gradient Descent	Gradient Descent
출력 해석	단순 분류	선형 점수	확률
특징	가장 초기 신경망 모델	퍼셉트론 개선 모델	확률 기반 분류 모델

3.3.3 아달린 구현을 로지스틱 회귀 알고리즘으로 변경

3.3.4 사이킷런을 사용하여 로지스틱 회귀 모델 훈련

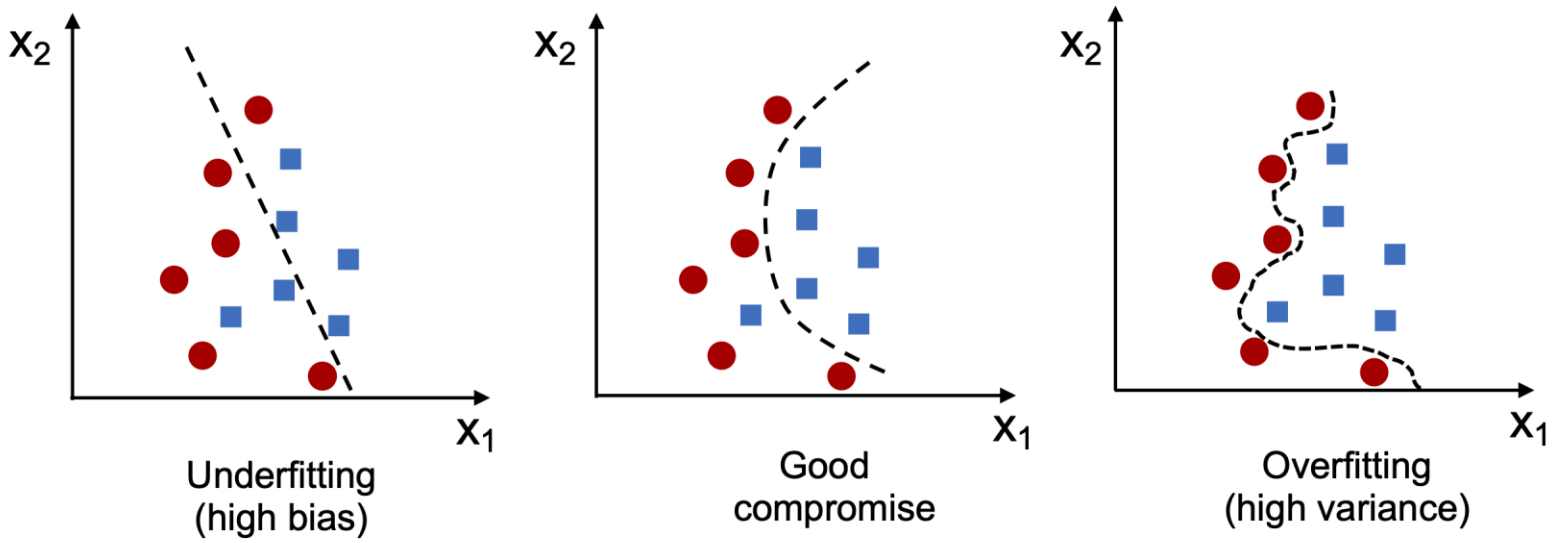
```
# 로지스틱 회귀 모델 훈련
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(
    C=100.0,
    solver='lbfgs',
    multi_class='ovr'
)
lr.fit(X_train_std, y_train
)
```

solver	특징
lbfgs	기본, 안정적
liblinear	작은 데이터
sag	대규모 데이터
saga	L1 규제 가능

방식	설명
ovr	One vs Rest
multinomial	Softmax

3.3.5 규제를 사용하여 과대적합 피하기

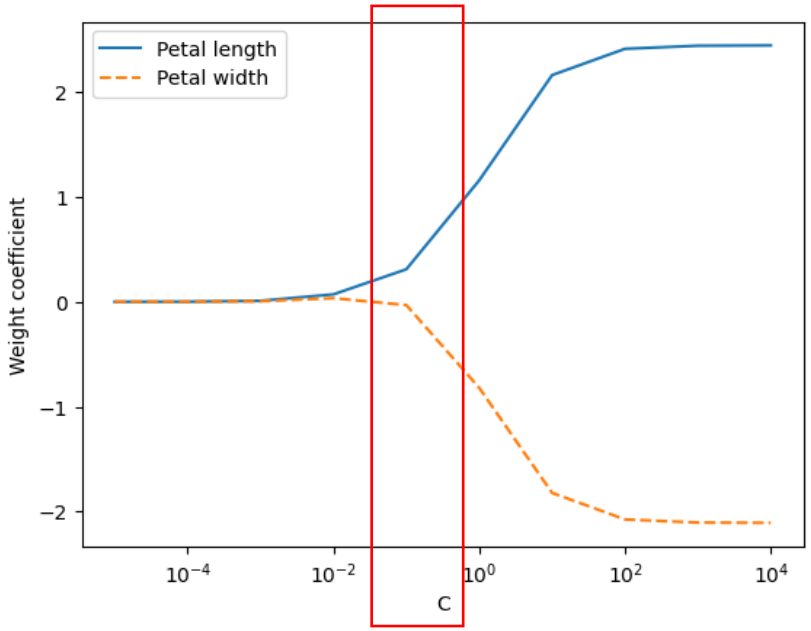
- 과대적합(overfitting)은 자주 발생하는 문제
  - 일반화 되지 않는 현상



C: 모델이 얼마나 선을 자유롭게 그리는지

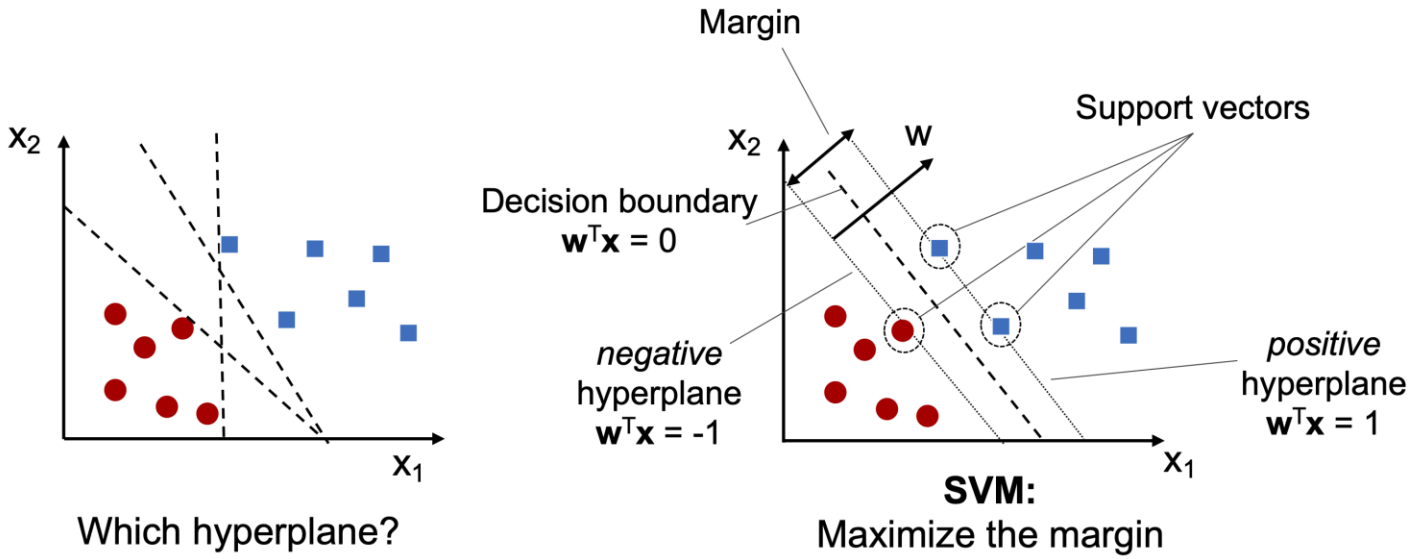
3.3.5 규제를 사용하여 과대적합 피하기

- C = 정규화 강도의 역수
  (Inverse Regularization Strength)
- 정답은 없고, 0.001, 0.01, 0.1, 1, 10, 100



3.4 서포트 벡터 머신을 사용한 최대 마진 분류

- 서보트 벡터 머신(Support Vector machine, SVM) 마진을 최대화 하는 방법



3.4.1 최대 마진

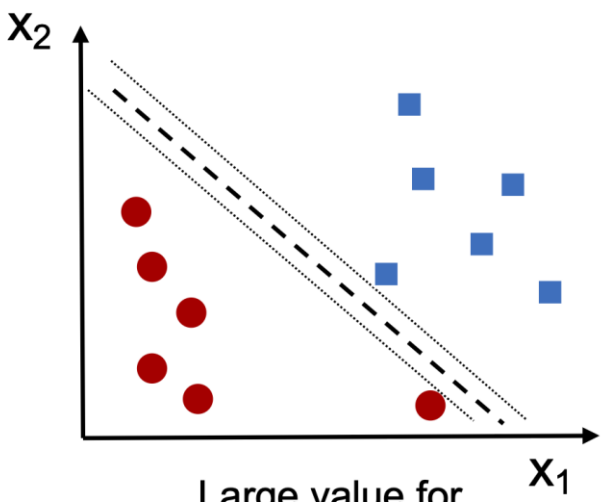
- 일반화 오차가 낮아지는 경향이 있기 때문
- SCM의 주요 개념은 비교적 간단하지만, 이면에 있는 수학은 상당히 복잡

3.4.2 슬랙 변수를 사용하여 비선형 분류 문제 다루기

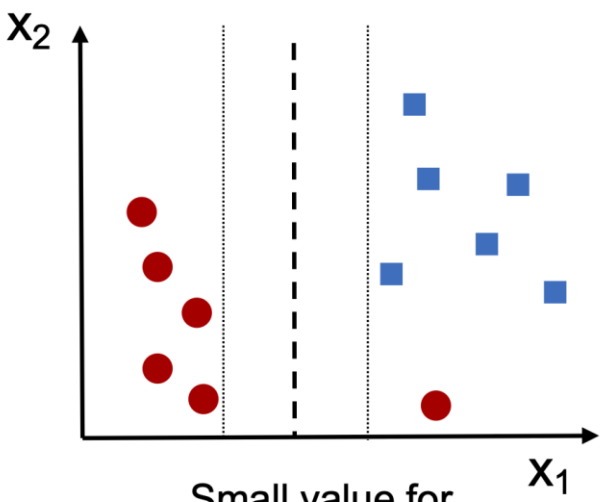
```
# SVM 모델 훈련
from sklearn.svm import SVC
svm = SVC(
    kernel='linear',
    C = 1.0,
    random_state=1
    )
svm.fit(X_train, y_train)
```

3.4.2 슬랙 변수를 사용하여 비선형 분류 문제 다루기

- C 값이 크면 오차에 대한 손실이 커지고
- C 값이 작아지면 분류 오차에 덜 엄격



Large value for parameter C



Small value for parameter C

3.4.3 사이킷런의 다른 구현

```
from sklearn.linear_model import SGDClassifier

ppn = SGDClassifier(loss='perceptron')
lr = SGDClassifier(loss='log')
svm = SGDClassifier(loss='hinge')
```

쉬는 시간

3. 사이킷런을 타고 떠나는 머신 러닝 분류 모델 투어

3.5 커널 SVM을 사용하여 비선형 문제 풀기
3.6 결정 트리 학습
3.7 K-최근접 이웃: 게으른 학습 알고리즘
3.8 요약

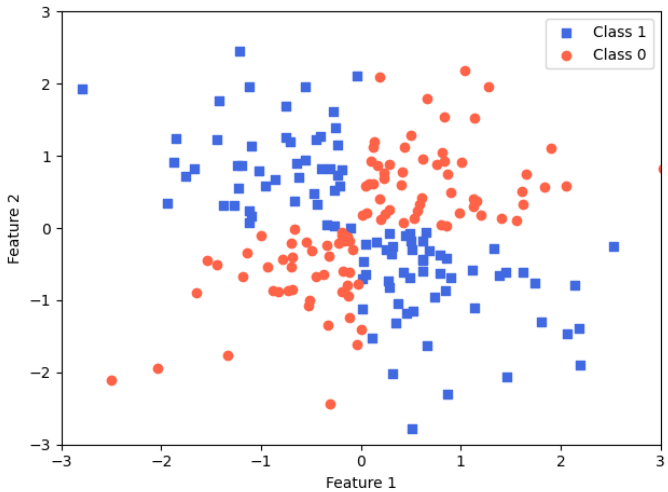
3.5 커널 SVM을 사용하여 비선형 문제 풀기

- 머신러닝 기술자 사이에서 SVM이 인기가 높은 이유
  - 비선형 분류 문제를 풀기 위해 커널 방법을 사용할 수 있기 때문

- 데이터셋
  - logical_xor 함수를 사용하여 XOR 형태 생성
  - 100개의 샘플은 클래스 레이블 1로 할당
  - 나머지 100개의 샘플은 클래스 레이블 -1로 할당

3.5 커널 SVM을 사용하여 비선형 문제 풀기

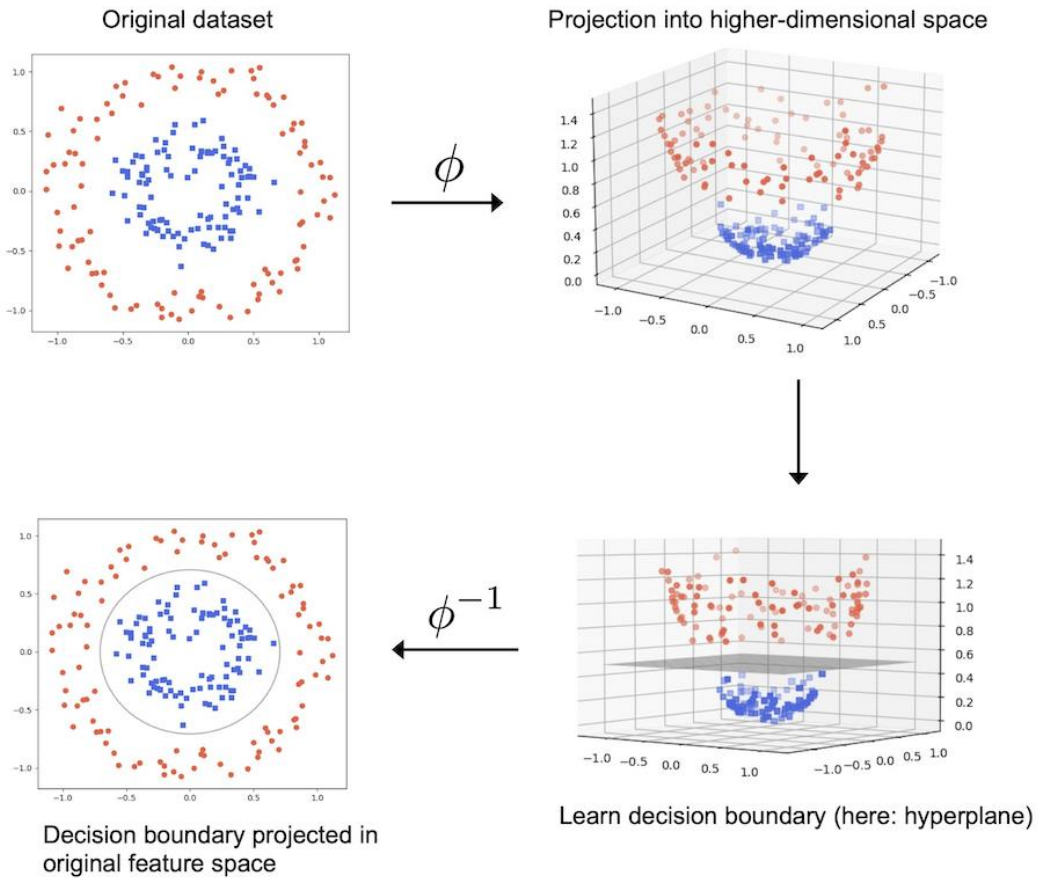
```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)
X_xor = np.random.randn(200, 2)
y_xor = np.logical_xor(
        X_xor[:, 0] > 0,
        X_xor[:, 1] > 0
)
y_xor = np.where(y_xor, 1, 0)
```



x1>0	x2>0	XOR
True	True	False
True	False	True
False	True	True
False	False	False

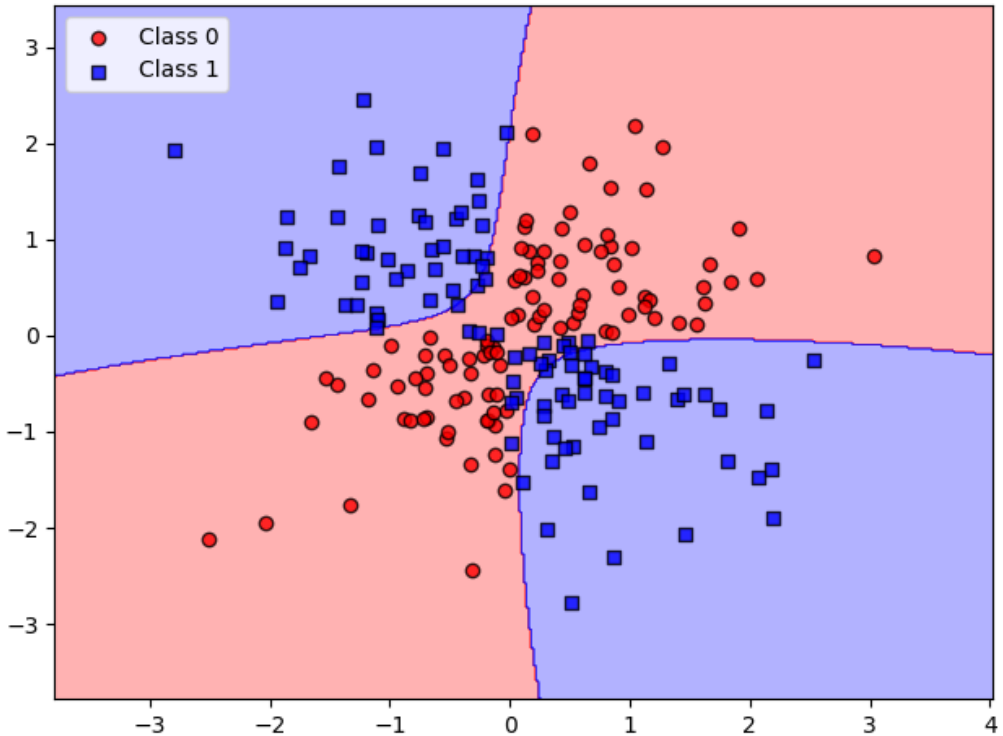
3.5.1 선형적으로 구분되지 않는 데이터를 위한 커널 방법

- 2차원 데이터를 새로운 3차원 공간에 투영



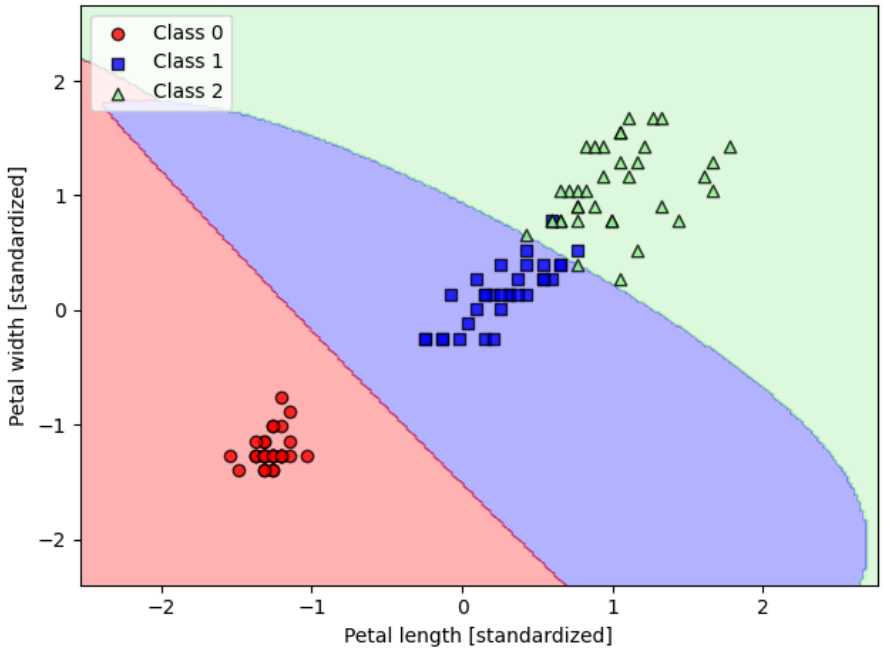
3.5.2 커널 기법을 사용하여 고차원 공간에서 분할 초평면 찾기

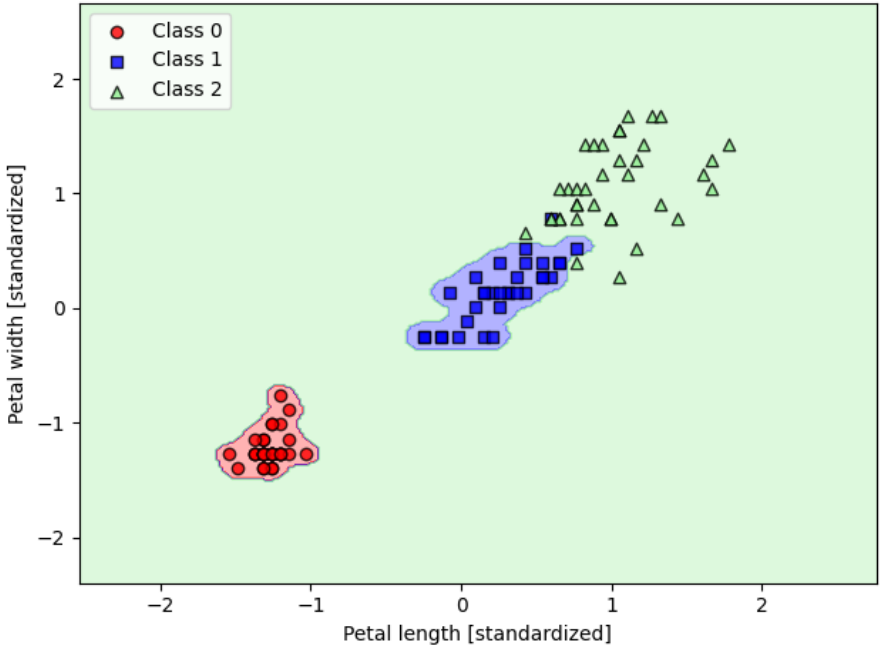
```
svm = SVC(kernel='rbf', random_state=1, gamma=0.10, C=10.0)
svm.fit(X_xor, y_xor)
```



3.5.2 커널 기법을 사용하여 고차원 공간에서 분할 초평면 찾기

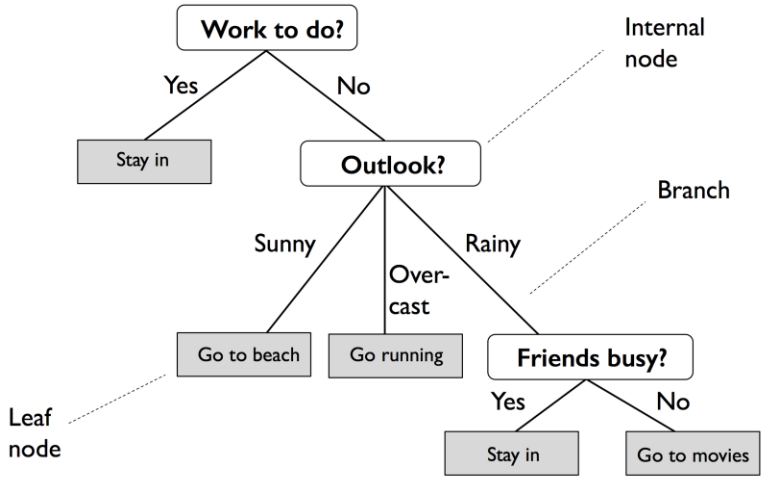
```
svm = SVC(kernel='rbf', random_state=1, gamma=0.2, C=1.0)
svm.fit(X_train_std, y_train)
svm = SVC(kernel='rbf', random_state=1, gamma=100, C=1.0)
svm.fit(X_train_std, y_train)
```





3.6 결정 트리 학습

- 설명이 중요할 때 아주 유용한 모델

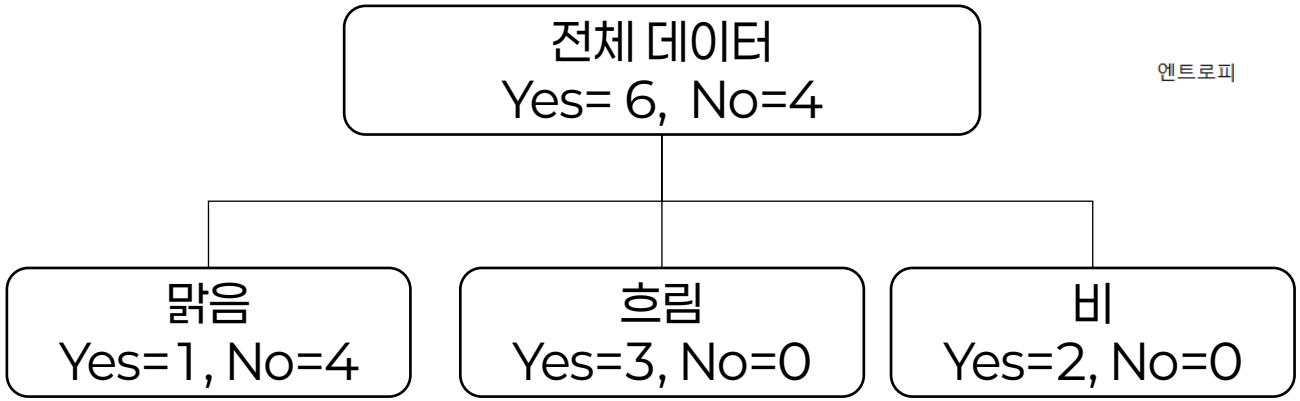


알고리즘	기준	특징
ID3 (초기 오픈)	Entropy / Information Gain	초기 알고리즘
C5 (상용)	Gain Ratio	ID3, C4.5 개선
CART	Gini	현재 가장 많이 사용

Classification And Regression Trees

3.6.1 정보 이득 최대화: 자원을 최대로 활용

- 정보 이득



확률

$$P(Yes) = 6/10 = 0.6$$

$$P(No) = 4/10 = 0.4$$

엔트로피

$$Entropy(S) = -(0.6\log_2 0.6 + 0.4\log_2 0.4)$$

$$= -(0.6(-0.737) + 0.4(-1.322))$$

$$= 0.971$$

$$Entropy_{after}$$

$$= (5/10 \times 0.722) + (3/10 \times 0) + (2/10 \times 0)$$

$$= 0.361$$

$$P(Yes) = 1/5 = 0.2$$

$$P(No) = 4/5 = 0.8$$

$$Entropy(\text{맑음}) = -(0.2\log_2 0.2 + 0.8\log_2 0.8)$$

$$= -(0.2(-2.322) + 0.8(-0.322))$$

$$= 0.722$$

$$InformationGain$$

$$= Entropy(parent) - Entropy(after)$$

$$= 0.971 - 0.361$$

$$= 0.610$$

3.6.2 결정 트리 만들기

```
from sklearn.tree import DecisionTreeClassifier

tree_model = DecisionTreeClassifier(
                    criterion='gini',
                    max_depth=4,
                    random_state=1
)
tree_model.fit(X_train, y_train)
```

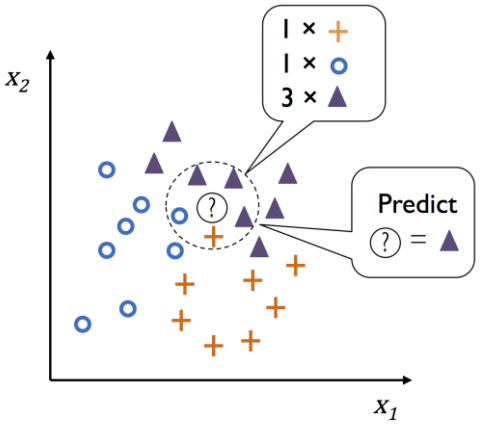
3.6.3 랜덤 포레스트로 여러 개의 결정 트리 연결

- 앙상블: 여러 모델을 결합하는 전체 방법

구분(방법론)	배깅 (Bagging)	부스팅 (Boosting)
학습 방식	모델을 **독립적으로 병렬 학습**	모델을 **순차적으로 학습**
데이터 처리	Bootstrap 샘플링	이전 모델 오류에 가중치 부여
목표	분산 감소	편향 감소
대표 알고리즘	Random Forest	AdaBoost, Gradient Boosting, XGBoost

3.7 k-최근접 이웃: 게으른 학습 알고리즘

- KNN은 전형적인 게으른 학습기
  - 훈련 데이터에서 판별 함수를 학습하는 대신, 훈련 데이터셋을 메모리에 저장하기 때문
- KNN 알고리즘
  1. 숫자 k와 거리 측정 기준을 선택
  2. 분류하려는 샘플에서 k개의 최근접 이웃을 찾음
  3. 다수결 투표를 통해 클래스 레이블 할당



3.7 k-최근접 이웃: 게으른 학습 알고리즘

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(
                    n_neighbors=5,
                    p=2,
                    metric='minkowski'
)
knn.fit(X_train_std, y_train)
```

metric	설명
euclidean	유클리드 거리
manhattan	맨해튼 거리
minkowski	일반화 거리
chebyshev	최대 차이
cosine	코사인 거리
hamming	해밍 거리

3.8 요약

- 퍼셉트론, 아달린, 로지스틱 회귀
- 서포트 벡터 머신
- 결정 트리
- k-최근접 이웃

쉬는 시간